Tecnologias de Mercado

2025/2026

Programação em Java

Versão 1.0.0

Última atualização 16/02/2026

Docentes:

Alexandre Pita (aapita@ualg.pt)

Daniel Clímaco (dsclimaco@ualg.pt)

Índice

0. Sobre este documento 4

1. Introdução ao Java 4

1.1. A linguagem 4

O que é programação? 4

Linguagens compiladas e linguagens interpretadas 5

1.2. Introdução à programação em Java 8

O que é um programa 8

O primeiro problema 9

Modificar o output 10

Números 10

Aritmética básica 11

Executar várias impressões 12

Parênteses 12

Concatenação de texto 13

Concatenação entre texto e números 14

Tipos de dados 15

Checkpoint #1 16

Variáveis 16

Identificadores e literais 18

Entrada de dados 19

Checkpoint #2 21

Instrução *if-else* 22

Comentários 27

Instrução *switch* 30

Booleans 38

Operadores de comparação 39

Checkpoint #3 40

Operadores lógicos 41

Divisão e resto 47
Checkpoint #4 53
Ciclo while 53
Ciclo *do while* 58
Ciclo *for* 60
Instrução *continue* 64
Instrução *break* 67
Checkpoint #5 69
Introdução à Programação Orientada a objetos 70
Classes 70
Métodos 73
Parâmetros de métodos 75
Tipo de retorno de métodos 76
Construtores 77
Objetos vs primitivos 78
Stack vs heap 78
Garbage Collection 79
Referência 80
Tipos de dados 80
Primitivos 80
Não-primitivos 80
Operadores 80
Aritmética 80
Comparação 81
Lógica 81
Outros 81
Controlo 82
Instrução *if* 82
Ciclo *while* 82

Ciclo *do while* 82
Instruções break e continue 82
Input/Output 82
Ler input 82
Imprimir output 83
Métodos 83
Modificadores 83
Sem retorno 83
Com retorno 84

0. Sobre este documento

Este documento divide-se em 2 grandes partes:

- **Guia de aprendizagem** – Resumo das aulas que ensina os conceitos essenciais dados na aula. Está exposto de forma sequencial das aulas e de forma guiada. Podes usar para reveres e estudares em casa ou caso tenhas faltado à aula.
- **Referência** – Conceitos essenciais do Java estruturados de forma concisa. Podes usar como cábula (para estudo, exercícios ou trabalhos) ou referência de conceitos.

1. Introdução ao Java

1.1. A linguagem

O que é programação?

Programação é muito mais do que apenas escrever código. Escrevemos programas para resolver problemas. Antes de se quer pensar numa solução, temos de ter a certeza de que percebemos bem o problema. Só depois disso começamos a pensar em soluções para o resolver. Depois de termos uma ideia temos de pensar como implementá-la em código.

O que é programação

- Resolução de problemas
- Planear uma solução
- Escrever o código
- Testar e depurar
- Manutenção e melhorias
- Comunicação e colaboração

Existe uma frase popular de Abraham Lincoln que enfatiza a importância de planear antes de realizar uma tarefa. Neste caso particular, devemos pensar no problema, pensar numa solução, pensar em como implementar e só depois implementar.

“Give me six hours to chop down a tree and I will spend the first four sharpening the axe.” – Abraham Lincoln

Linguagens compiladas e linguagens interpretadas

Computadores não entendem código escrito por humanos, eles apenas executam instruções em binário, conhecido como código de máquina. Existem 2 grandes tipos de linguagens de programação: linguagens compiladas e linguagens interpretadas.

Linguagem compilada

O código-fonte (código em texto escrito por um humano) é enviado para um programa chamado o compilador, que transforma o código-fonte em código de máquina, em formato binário. Este código normalmente vem dentro de um ficheiro executável que varia consoante o sistema operativo (por exemplo .exe no Windows ou .out em macOS).

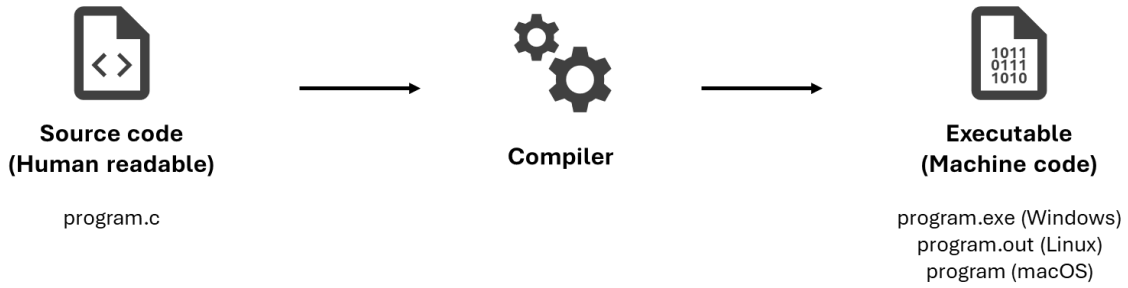


Figura 1 - Processo de uma linguagem compilada

Linguagem interpretada

O código-fonte é enviado para um programa chamado o interpretador que irá interpretar o código e executar as instruções. A desvantagem destes tipos de linguagens é que são menos eficientes, demoram mais tempo a executar.

Figura 2 - Processo de uma linguagem interpretada

Java

A linguagem Java junta o melhor dos dois mundos e, apesar de ser considerada compilada, é uma linguagem intermédia entre compilada e interpretada. O código-fonte Java é enviado para o compilador que transforma o código em uma linguagem intermédia entre alto nível e código máquina, chamado de **Bytecode**. Este Bytecode é contido dentro de um ficheiro .class, que para ser executado tem de ser interpretado por um programa chamado **Java Virtual Machine (JVM)**.

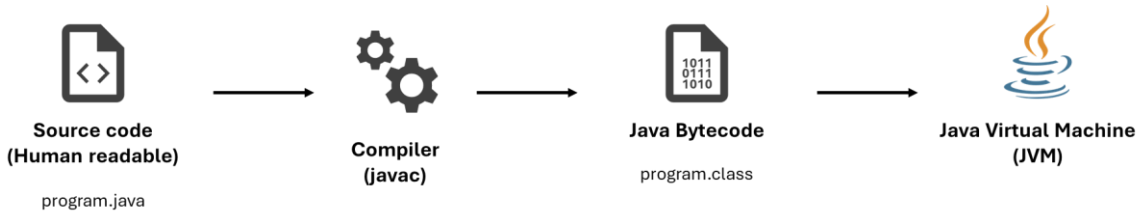


Figura 3 - Processo da linguagem Java

Execução de programas Java

Esta abordagem é mais rápida do que uma linguagem interpretada, mas menos do que uma compilada. Contudo, tem uma vantagem importante que é o facto de o programa emitido não depender do sistema operativo. Um utilizador que pretenda executar um programa Java, apenas necessita de ter instalado o **Java Runtime Environment (JRE)**, que nada mais é do que um programa que corre programas Java. Este contém, além de outras funcionalidades e recursos, a JVM, necessária para rodar programas Java. Isto facilita a nossa vida, porque não nos precisamos preocupar em emitir executáveis para cada tipo de sistema operativo. Isto deu origem ao slogan "Write once, run anywhere", da Sun Microsystems – escreve o código uma vez e executa em qualquer lugar.

Desenvolvimento de programas Java

Já sabemos que para executar programas Java precisamos do JRE, porém para desenvolvermos programas Java precisamos de algo chamado o **Java Development Kit (JDK)**, que consiste num conjunto de ferramentas que permitem criar programas em Java. O JDK já contém também a JVM para executar programas.

1.2. Introdução à programação em Java

Código (ou código-fonte) é texto escrito numa linguagem de programação, que define uma série de instruções que são interpretadas ou compiladas para serem executadas por um computador. Os fundamentos da programação estão fortemente ligados à lógica e à matemática.

O que é um programa

Um programa, fundamentalmente, recebe dados de entrada (input) e produz dados de saída (output). Input pode ser texto, um ficheiro, cliques do rato, etc. Output pode ser texto, um ficheiro, uma imagem, etc.

Figura 4 - Fluxo de dados de um programa

Por exemplo, um programa "Odd or even" recebe como dados de entrada um número e devolve como dados de saída "Even", caso o número seja par, ou "Odd", caso seja ímpar.

Figura 5 - Exemplo de programa (número par)

Figura 6 - Exemplo de programa (número ímpar)

O primeiro problema

No mundo da programação, um primeiro exercício costuma ser **imprimir "Hello, world!" na consola**. Este será o nosso primeiro problema. Como aprendemos atrás, antes de começarmos a pensar na implementação ou se quer na solução, temos de primeiro entender bem o problema e depois sim planear uma solução.

Entender o problema

Algumas questões que podemos fazer:

- O que se pretende fazer? – Imprimir uma mensagem na consola
- Qual é a mensagem que queremos imprimir? "Hello, world!"

Portanto, o objetivo é que, ao executar o programa, apareça na consola a mensagem "Hello, world!".

Planear e implementar uma solução

Por defeito o output de um programa é enviado para a consola. Sabendo isso, queremos então desenvolver uma solução que envie a mensagem pretendida para o output. A solução para este problema é algo como o seguinte:

```
public class Main {
      public static void main(String[] args) {
            System.out.println("Hello, world!");
      }
}
```

Este código imprime na consola a mensagem "Hello, world!". Para vermos isso a acontecer temos de primeiro compilar o programa e em seguida executá-lo. Cria um ficheiro com o nome "Main.java" e coloca nele o código acima. Para compilar executamos primeiro `javac Main.java`. Depois, para executar o programa executamos `java Main`.

O resultado na consola será:

```
Hello, world!
```

Todos os programas Java devem ter uma classe com um método *main* escrito da forma como foi apresentada acima. Não precisas entender ou decorar toda a sintaxe por agora, iremos abordar isto em maior detalhe mais à frente.

Modificar o output

O que acontecerá se nós mudarmos o texto "Hello, world!" para algo como por exemplo "Hi Java!"?

```
public class Main {
	public static void main(String[] args) {
		System.out.println("Hi Java!");
	}
}
```

O resultado na consola passará a ser:

```
Hi Java!
```

Números

Esta função não permite apenas imprimir texto, como também permite imprimir números.

```
public class Main {
	public static void main(String[] args) {
		System.out.println(25);
	}
}
```

Resultado na consola:

```
25
```

Aritmética básica

O que acontecerá se em vez de 25, escrevermos uma conta como por exemplo 1 + 2?

```
public class Main {
	public static void main(String[] args) {
		System.out.println(1 + 2);
	}
}
```

Resultado na consola:

```
3
```

Ele imprime o resultado da operação! De facto, podemos usar diversos operadores aritméticos. Abaixo estão 3 dos principais.

- Adição – `+`
- Subtração – `-`
- Multiplicação – `*`

O exemplo abaixo inclui 2 operadores distintos.

```
public class Main {
	public static void main(String[] args) {
		System.out.println(4 + 3 * 2);
	}
}
```

Resultado na consola:

```
10
```

Executar várias impressões

Podemos também imprimir várias vezes.

```
public class Main {
      public static void main(String[] args) {
            System.out.println(4 + 3 * 2);
            System.out.println(3 * 4 + 3);
      }
}
```

Resultado na consola:

```
10
15
```

Parênteses

Tal como na matemática, podemos usar parênteses para agrupar expressões.

```
public class Main {
      public static void main(String[] args) {
            System.out.println(4 + 3 * 2);
            System.out.println((4 + 3) * 2);
      }
}
```

Resultado na consola:

```
10

14
```

Concatenação de texto

Para juntar textos, usamos na mesma o operador `+`, mas neste caso em vez de adição chama-se de **concatenação**, que nada mais é do que a junção de 2 textos. Observa o exemplo abaixo.

```
public class Main {
      public static void main(String[] args) {
            System.out.println("Olá" + "mundo");
      }
}
```

Resultado na consola:

```
Olámundo
```

Observa que o resultado na consola tem as palavras juntas sem espaço. É importante notar que na programação é tudo muito literal e temos de "dizer ao computador" exatamente o que queremos. Neste caso, se calhar nós esperávamos que fosse imprimir "Olá mundo", quando na verdade imprimiu "Olámundo". A solução para isto é muito simples, ou adicionamos um espaço após "Olá" ou adicionamos um espaço antes de "mundo"!

```
public class Main {
      public static void main(String[] args) {
            System.out.println("Olá " + "mundo");
      }
}
```

Resultado na consola:

```
Olá mundo
```

Concatenação entre texto e números

Já sabemos imprimir texto (ex.: “Olá, mundo!”) e números (ex.: 25). E se quisermos juntar os dois?

```
public class Main {
      public static void main(String[] args) {
            System.out.println("Idade: " + 25);
      }
}
```

Resultado na consola:

```
Idade: 25
```

Funciona! E se quisermos juntar texto com o resultado de uma operação?

```
public class Main {
      public static void main(String[] args) {
            System.out.println("Soma: " + 2 + 4);
      }
}
```

Resultado na consola:

```
Soma: 24
```

Imprimiu “Soma: 24” em vez de “Soma: 6”. Porque é que isto acontece?

Se formos analisar passo a passo:

1. `"Soma: " + 2` → `"Soma: 2"`
2. `"Soma: 2" + 4` → `"Soma: 24"`

Como podemos ver, após a primeira operação (`"Soma: " + 2`), o texto concatena ao número 2. Na segunda operação, o texto que temos até ao momento (`"Soma:  + 2"`) concatena ao número 4 formando o texto final `"Soma: 24"`.

Como podemos resolver isto?

Podemos separar o texto dos números, agrupando os números com parênteses.

```
public class Main {
      public static void main(String[] args) {
            System.out.println("Soma: " + (2 + 4));
      }
}
```

Resultado na consola:

```
Soma: 6
```

Assim já temos o resultado esperado!

Tipos de dados

Até ao momento, chamámos o que está entre aspas de texto e números de, bem... números. O que não está incorreto, mas na programação estes tipos de dados têm cada um, um nome.

Texto chamamos de **string**.

Números podem ter vários tipos, mas por agora vamos focar no mais simples, que é o **integer**. Como o próprio nome indica, representa um número inteiro.

Portanto, **strings representam texto**, **integers (ou inteiros) representam números inteiros**.

Checkpoint #1

Revê o que aprendeste

- Enviar **output** para a consola:
  - `System.out.println("Hi Java!");`
- Aritmética:
  - **Adição** (`+`), **subtração** (`-`) e **multiplicação** (`*`)
- Tipos de dados
  - ***String*** – texto
  - ***Integer*** – números inteiros
- Imprimir texto e números
  - `System.out.println("Resultado: " + ((5 + 3) * 2));`

Variáveis

Em programação, existe um conceito fundamental que são as variáveis. Uma **variável** é como se fosse uma caixa que guarda um dado (valor). Este dado pode ser lido ou modificado. Por exemplo, podemos criar uma variável que guarda a idade de uma pessoa. Ao criar uma variável em Java, temos de especificar o tipo e em seguida atribuímos um valor.

Qual será o tipo de dado mais adequado para guardar a idade de uma pessoa?

Seria o tipo *integer*. O seguinte código cria uma variável do tipo *integer* e atribui o valor 25.

```
public class Main {
      public static void main(String[] args) {
            int age = 25;
      }
}
```

Este código por si só não faz nada, pelo menos que seja visível. Apenas criámos uma variável com valor 25 e não imprimimos nada.

Podemos recorrer ao conhecimento anterior, para imprimir o valor da variável na consola.

```
public class Main {
      public static void main(String[] args) {
            int age = 25;
            System.out.println(age);
      }
}
```

Resultado na consola:

```
25
```

Ele imprimiu o valor da variável na consola!

Podemos alterar o valor da variável da seguinte forma.

```
public class Main {
      public static void main(String[] args) {
            int age = 25;
            age = 30;
            System.out.println(age);
      }
}
```

Resultado na consola:

```
30
```

O valor na consola passou a ser o novo valor da variável (30).

Repara que aqui o sinal de igual foi usado para alterar o valor da variável. Quando declaramos a variável (`int age = 25`), atribuímos um **valor inicial** de 25. Quando alterámos o valor da variável (`age = 30`), atribuímos um **novo valor** de 30. Este sinal de igual, ao contrário da matemática, atribui um valor a uma variável. Na matemática, geralmente, este sinal seria usado para comparar ou afirmar se `age` igual é a 30. Portanto, na programação, este sinal é conhecido como **operador de atribuição**, pois, como o próprio nome indica, atribui um (novo) valor a uma variável, isto é, altera o valor da variável.

Assim como conseguimos realizar operações entre números, também podemos fazê-lo com variáveis. Vamos somar 10 à nossa variável.

```
public class Main {
      public static void main(String[] args) {
            int age = 25;
            age = age + 10;
            System.out.println(age);
      }
}
```

Resultado na consola:

```
35
```

Como esperado, o valor na consola é o resultado da soma. As regras de aritmética básica da matemática aplicam-se aqui. Podemos colocar um sinal de menos atrás de uma variável para obter o seu simétrico. Se tivermos uma variável `x` com valor 10, `-x` será -10.

Identificadores e literais

Um **identificador** nada mais é do que o nome de algo, por exemplo de uma variável. Vimos atrás as variáveis `age` e `x`, nesse caso `age` e `x` são identificadores, que representam variáveis.

Nota que, apesar de nos exemplos anteriores termos feito `System.out.prinln(age);`, o programa não imprimiu "age", mas sim o valor da variável `age`. Isto é importante notar, pois noutros exemplos em que usámos aspas para imprimir texto (`System.out.println("Hello, world!");`), ele imprime **literalmente** o que escrevemos, pois `"Hello, world!"` é uma string, e é o que chamamos de um valor **literal** (neste caso uma *string* literal). Nos outros exemplos, quando imprimimos um número (`System.out.println(25);`), nós escrevemos o número e ele imprime **literalmente** o número, pois `25` é um inteiro literal. Em contraste, os identificadores de variáveis não são literais, são o nome de uma variável que contêm um valor.

Regras de identificadores

- Podem conter letras, dígitos, *underscores* (_), e o símbolo do dólar ($)
- Têm de começar com uma letra, $ ou *underscore*
- São sensíveis a maiúsculas e minúsculas ("myVar" é diferente de "myvar")
- Não podem ser palavras reservadas (como *int* ou *if*)

Entrada de dados

Até agora sabemos imprimir dados na consola – **saída de dados**. Mas e se quisermos que o utilizador forneça dados ao programa? Vamos ver como podemos ler a **entrada de dados**.

Como podemos por exemplo pedir um número ao utilizador e depois imprimi-lo de volta?

```
import java.util.Scanner;

public class Main {
	public static void main(String[] args) {
		Scanner scanner = new Scanner(System.in);
		int x = sc.nextInt();
		System.out.println(x);
	}
}
```

Consola:

```
25
25
```

O primeiro número foi introduzido pelo utilizador, o segundo número é o que foi impresso.

Este número pode por exemplo ser a idade do utilizador. Podemos pedir-lhe a idade e imprimir quanto será a idade dele no próximo ano.

```
import java.util.Scanner;

public class Main {
	public static void main(String[] args) {
		Scanner scanner = new Scanner(System.in);
```

```
            System.out.println("Enter your age:");
            int age = scanner.nextInt();
            System.out.println("Your age next year will be " + (age
+ 1));
      }
}
```

Consola:

```
Introduz a tua idade:
25
A tua idade no próximo ano será 26
```

Outro exemplo, pode ser pedir o nome e cumprimentar com o mesmo.

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.println("What's your name?");
        String name = scanner.nextLine();
        System.out.println("Hello, " + name);
    }
}
```

Consola:

```
What’s your name?
John
Hello, John
```

Checkpoint #2

Revê o que aprendeste

- Variáveis:
  - Definir
    - Exemplo: `int x;`
  - Definir e inicializar
    - Exemplo: `int x = 0;`
  - Alterar o valor
    - Exemplo: `x = 5;`
- Identificadores: Nomes de coisas, por exemplo variáveis.
- Literais: Valores como `"Hello"`, `"Hello World"` e `25`.
- Ler input
  - `Scanner scanner = new Scanner(System.in);`
  - Ler um inteiro (*integer*): `scanner.nextInt()`
  - Ler uma linha (*string*): `scanner.nextLine()`

Instrução *if-else*

Imagina um programa em que o utilizador introduz uma idade e o programa diz se o utilizador é um adulto ou não.

Muitas vezes um programa tem de decidir entre diferentes caminhos.

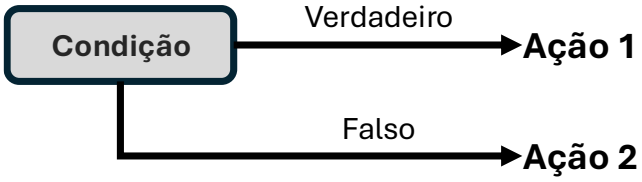


Figura 7 - Fluxo de programa com condição

O programa que verifica se um utilizador é adulto, pode ser representado da seguinte forma, esquematicamente.

Figura 8 - Fluxo de exemplo de programa "Is an adult?"

Recebemos a idade como input e imprimimos “Is adult” caso seja maior ou igual a 18, caso contrário imprime “Is not adult”.

```java
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.println("What’s your age?");
        int age = scanner.nextInt();

        if (age >= 18) {
            System.out.println("Adult");
        } else {
            System.out.println("Minor");
        }
    }
}
```

Consola:

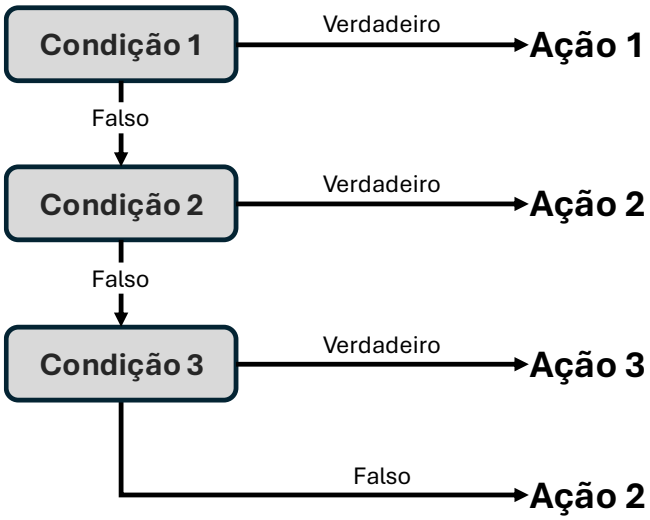
```
What’s your age?
25
Adult
```

Se introduzirmos uma idade inferior a 18:

```
What’s your age?
16
Minor
```

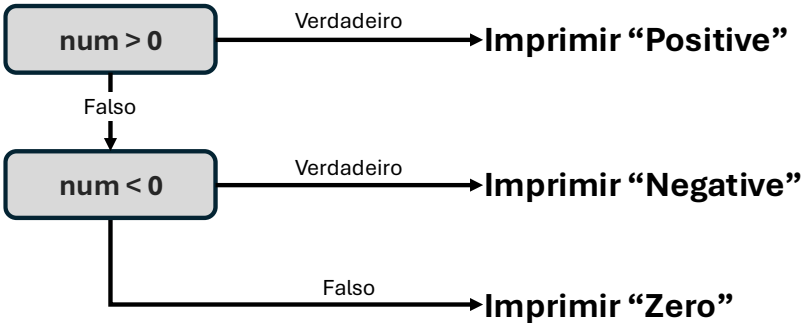
Portanto, a instrução *if-else* permite-nos escolher qual pedaço de código executar.

Mas, o *if* pode ser expandido com mais caminhos possíveis.



Imagina um programa "Number sign" que imprime o sinal de um número.

- Maior que 0 – imprime "Positive"
- Menor que 0 – imprime "Negative"
- Caso contrário – imprime "Zero"



Se formos ver, podíamos fazer o seguinte

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.println("Enter an integer");
        int number = scanner.nextInt();

        if (number > 0) {
            System.out.println("Positive");
        } else {
            if (number < 0) {
                System.out.println("Negative");
            } else {
                System.out.println("Zero");
            }
        }
    }
}
```

Mas, como podes ver o código pode-se tornar confuso e difícil de ler. Felizmente, há uma forma muito simples de resolver isto. Podemos retirar as chavetas após o *else* e escrever seguido, da seguinte forma.

```java
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.println("Enter an integer");
        int number = scanner.nextInt();

        if (number > 0) {
            System.out.println("Positive");
        } else if (number < 0) {
            System.out.println("Negative");
        } else {
            System.out.println("Zero");
        }
    }
}
```

O código faz o mesmo, mas está mais limpo.

Comentários

Quando o código começa a ficar maior ou mais complexo, é boa prática incluir comentários.

Observa o exemplo abaixo que implementámos anteriormente.

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("What's your age?");
        int age = scanner.nextInt();

        if (age >= 18) {
            System.out.println("Adult");
        } else {
            System.out.println("Minor");
        }
    }
}
```

Alguém que olha pela primeira vez para o código pode não entender logo o que faz, daí comentários ajudarem.

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Ask for age
        System.out.println("What's your age?");
        int age = scanner.nextInt();

        // Check if user is an adult
        if (age >= 18) {
            System.out.println("Adult");
        } else {
            System.out.println("Minor");
        }
    }
}
```

Os comentários não só ajudam a entender melhor o código, como também permitem dar mais contexto sobre o problema e estruturar melhor o código.

Mas é importante **não encher o código de comentários**. Os comentários **devem ser complementares e não redundantes**.

Por exemplo, este comentário é **redundante**:

```
// Read an integer from the console
int age = scanner.nextInt();
```

Um comentário mais **útil** e **adequado** seria:

```
// Read user's age to determine if they are an adult
int age = scanner.nextInt();
```

Não expliques coisas demasiado óbvias ou literais. Também não deves comentar cada linha de código. Deves comentar blocos de código que definem uma lógica.

Também é possível escrever comentários de múltiplas linhas. Por exemplo:

```
import java.util.Scanner;

/*
  This program prints whether a user has access to an event.
  To have access, the user must be an adult (18+).
*/
public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Ask for age
        System.out.println("What's your age?");
        int age = scanner.nextInt();

        // Check if user is an adult
        if (age >= 18) {
            System.out.println("Access granted");
        } else {
            System.out.println("Access denied");
        }
    }
}
```

De modo geral, comentários de uma linha são melhores para explicações curtas para uma linha ou blocos pequenos. Comentários de múltiplas linhas são melhores para explicações mais extensas ou blocos de código mais complexos.

Mas é importante notar que explicações muito extensas e complexas provavelmente são mais indicadas para documentações e não comentários em código.

Instrução *switch*

Imagina agora um programa que, dado um número, imprime o dia da semana correspondente a esse número.

- 1 – Domingo
- 2 – Segunda-feira
- 3 – Terça-feira
- 4 – Quarta-feira
- 5 – Quinta-feira
- 6 – Sexta-feira
- 7 – Sábado

Queremos imprimir uma *string* consoante o número. Aprendemos já como fazer decisões usando a instrução *if-else*.

```java
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Enter a number");
        int number = scanner.nextInt();

        if (number == 1) {
            System.out.println("Sunday");
        } else if (number == 2) {
            System.out.println("Monday");
        } else if (number == 3) {
            System.out.println("Tuesday");
        } else if (number == 4) {
            System.out.println("Wednesday");
        } else if (number == 5) {
            System.out.println("Thursday");
        } else if (number == 6) {
            System.out.println("Friday");
        } else if (number == 7) {
            System.out.println("Saturday");
        } else {
            System.out.println("Invalid number");
        }
    }
}
```

Consola:

```
1
Sunday
```

```
4
Wednesday
```

```
8
Invalid number
```

Funciona como queremos! Mas podes notar que o código está um pouco repetitivo. Estamos constantemente a comparar a mesma variável com um número. Em Java, podemos usar uma instrução que simplifica e facilita este tipo de casos, que é a instrução ***switch***. O *switch* recebe um valor e definimos ações para cada caso.

```java
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Enter a number");
        int number = scanner.nextInt();

        switch (number) {
            case 1:
                System.out.println("Sunday");
                break;
            case 2:
                System.out.println("Monday");
                break;
            case 3:
                System.out.println("Tuesday");
                break;
            case 4:
                System.out.println("Wednesday");
                break;
            case 5:
                System.out.println("Thursday");
                break;
            case 6:
                System.out.println("Friday");
                break;
            case 7:
                System.out.println("Saturday");
                break;
            default:
                System.out.println("Invalid number");
        }
    }
}
```

O resultado é exatamente o mesmo, mas o código fica mais limpo.

Repara que cada `else if` virou um `case` e o `else` virou `default`. Neste caso temos então:

- `switch (number)` – queremos fazer comparações a `number`
- `case 1` – caso `number` seja igual a 1
- `case 2` – caso `number` seja igual a 2
- …
- `default` – caso `number` não seja igual a nenhum dos anteriores

Repara, ainda, que cada *case* tem um *break*, sem ele a execução continuaria para o próximo case (mesmo que o próximo case não seja verdadeiro). Vamos remover o *break* do case e ver o que acontece se introduzirmos 2.

```java
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Enter a number");
        int number = scanner.nextInt();

        switch (number) {
            case 1:
                System.out.println("Sunday");
                break;
            case 2:
                System.out.println("Monday");
                // break; <- break removed
            case 3:
                System.out.println("Tuesday");
                break;
            case 4:
                System.out.println("Wednesday");
                break;
            case 5:
                System.out.println("Thursday");
                break;
            case 6:
                System.out.println("Friday");
                break;
            case 7:
                System.out.println("Saturday");
                break;
            default:
                System.out.println("Invalid number");
        }    }
}
```

Consola:

```
2
Monday
Tuesday
```

Ele executou o *case* 3 mesmo o número não sendo 3. Depois terminou, pois chegou ao *break* do 3. Portanto o *break* é usado no *switch* para sair fora dele quando terminamos um *case*.

Agora podes-te perguntar: porque é que precisamos de colocar *break* em cada *case*? Existem razões históricas, mas também existem casos em que não ter *break* é útil.

Por exemplo, pegando no exemplo anterior, em vez de imprimirmos o dia da semana, queremos imprimir se o dado número do dia da semana corresponde a um fim de semana ou não.

```java
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Enter a number");
        int number = scanner.nextInt();

        switch (number) {
            case 2: // Monday
            case 3: // Tuesday
            case 4: // Wednesday
            case 5: // Thursday
            case 6: // Friday
                System.out.println("Weekday");
                break;
            case 1: // Sunday
            case 7: // Saturday
                System.out.println("Weekend");
                break;
            default:
                System.out.println("Invalid number");
        }
    }
}
```

Consola:

```
2
Weekday
```

```
1
Weekend
```

Booleans

A expressão que usámos anteriormente, `age >= 18`, significa "*age* é maior ou igual a 18". Isto é uma afirmação que pode ser tanto **verdadeira ou falsa**, consoante o valor da variável `age`. Se o valor de `age` for maior ou igual a 18, a afirmação é verdadeira, caso contrário é falsa. Em programação, `age >= 18` chama-se de uma **expressão booleana**. Que nada mais é do que uma expressão que resulta em `true` (verdadeiro) ou `false` (falso). Este resultado é o que se chama de um valor booleano (`true` ou `false`). Este é um tipo de dado chamado `boolean`.

Da mesma forma que podemos declarar variáveis do tipo *integer* (com `int`) e *string* (com `String`), podemos também declarar variáveis do tipo *boolean* (com `boolean`).

Já vimos antes, literais para números (ex.: 25) e *strings* ("Hello, world!"). Para booleanos também existem os literais `true` e `false`.

Podemos reescrever o código de antes, da seguinte forma.

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("What's your age?");
        int age = scanner.nextInt();

        boolean isAdult = age >= 18;

        if (isAdult) {
            System.out.println("Adult");
        } else {
            System.out.println("Minor");

        }
    }
}
```

Apesar do código ser ligeiramente diferente, o resultado é igual a antes.

Guardámos o valor da expressão `age >= 18` numa variável e de seguida usámo-la na instrução *if*.

Repara que o nome da variável é `isAdult`, que se traduz para “é adulto”. É comum em variáveis booleanas usar-se o prefixo “is”. Isto ajuda a pessoa que está a ler o código a perceber melhor e mias facilmente o tipo e o intuito da variável, que neste caso indica se é adulto ou não.

Operadores de comparação

Vimos em cima, que expressões booleanas são expressões que resultam num valor booleano (`true` ou `false`). No caso vimos até um exemplo de uma expressão booleana: `age >= 18`. Usámos o **operador de comparação** `>=`, que significa “maior ou igual”. Este operador vem direto da matemática, equivalente ao sinal de “maior ou igual”, ≥. Assim como na matemática, em Java temos vários operadores de comparação. Todos eles devolvem um valor booleano.

Operador	**Sintaxe**	**Significado**
Igual	`x == y`	x igual a y
Diferente	`x != y`	x diferente de y
Maior que	`x > y`	x maior que y
Maior ou igual	`x >= y`	x maior ou igual a y
Menor que	`x < y`	x menor que y
Menor ou igual	`x <= y`	x menor ou igual a y

Repara que o operador de comparação de igualdade usa 2 sinais de igual (`==`). Isto deve-se ao facto de que já existe o operador de atribuição (`=`).

Checkpoint #3

Revê o que aprendeste

- Instrução ***if-else***:

```
if (number > 0) {
    System.out.println("Positive");
} else if (number < 0) {
    System.out.println("Negative");
} else {
    System.out.println("Zero");
}
```

- **Comentários**:
    - 1 linha: `// Isto é um comentário de 1 linha`
    - Múltiplas linhas:

```
/*
  Isto é um comentário
  de múltiplas linhas.
*/
```

- Instrução ***switch***:

```
switch (number) {
    case 1:
        System.out.println("Sunday");
        break;
    case 2:
        System.out.println("Monday");
        break;
    // ...
    default:
        System.out.println("Invalid number");
}
```

- Tipo ***boolean***: Verdadeiro (`true`) ou falso (`false`).
- Operadores de **comparação**
    - `x == y` – x igual y
    - `x != y` – x diferente de y
    - `x > y` – x maior que y
    - `x >= y` – x maior ou igual a y
    - `x < y` – x menor que y
    - `x <= y` – x menor ou igual a y

Operadores lógicos

Problema de entrada num evento

Imagina que há um evento em que uma pessoa só tem acesso se for maior de idade e tiver um bilhete. Queremos escrever um programa que recebe a idade do utilizador (*integer*) e se ele tem bilhete (*boolean*). O programa no fim imprime "Access granted" caso cumpra os requisitos ou "Access denied", caso contrário.

Solucionar o problema

Vamos repartir o problema em diferentes partes:

1. **Qual é o input?**

   Um inteiro que representa a idade e um booleano que representa se possui bilhete.

2. **Quais são as condições do output?**

   O output é "Access granted" caso seja adulto e tenha bilhete ou "Access denied" caso contrário. Considera-se adulto alguém cuja idade é igual ou superior a 18.

3. **Como podemos ler o input?**

   Para ler o inteiro usamos `scanner.nextInt()` e para ler o booleano usamos `scanner.nextBoolean()`.

4. **Como podemos decidir o output?**

   Usamos a instrução *if-else* juntamente com expressões booleanas. Como temos 2 condições, temos de juntar ambas.

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("What's your age?");
        int age = scanner.nextInt();

        System.out.println("Do you have a ticket?");
        boolean hasTicket = scanner.nextBoolean();

        if (age >= 18 && hasTicket) {
            System.out.println("Access granted");
        } else {
            System.out.println("Access denied");
        }
    }
}
```

Consola:

```
What’s your age?
25
Do you have a ticket?
true
Access granted
```

```
What's your age?
20
Do you have a ticket?
false
Access denied
```

Consola:

```
What's your age?
15
Do you have a ticket?
true
Access denied
```

Testa com diferentes valores e analisa o resultado.

Em Java, usamos o operador `&&` para juntar duas condições, e significa que ambas as condições têm de ser verdadeiras. Neste caso a idade tem de ser maior ou igual a 18 e `hasTicket` tem de ser verdadeiro.

Nota que a segunda condição podia ter sido escrita da forma `hasTicket == true`, porém isso seria desnecessário, pois a variável `hasTicket` é do tipo boolean e a instrução *if* valida a veracidade de uma expressão. O valor de uma variável *boolean* já é um valor *booleano* (`true` ou `false`), portanto comparar com `true` seria redundante.

Problema de descontos em bilhetes

Imagina agora, que para o mesmo evento, uma pessoa pode ter um desconto se for sénior (idade igual ou superior a 65) ou se tiver um cupão de desconto. Queremos escrever um programa que recebe a idade do utilizador (*integer*) e se ele tem um cupão de desconto (*boolean*). O programa no fim imprime "Discount applied" caso cumpra os requisitos para ter desconto ou "No discount", caso contrário.

Novamente, vamos repartir o problema em diferentes partes:

1. **Qual é o input?**

   Um inteiro que representa a idade e um booleano que representa se possui um cupão de desconto.

2. **Quais são as condições do output?**

   O output é “Discount applied” caso seja sénior ou tenha cupão de desconto, ou “No discount” caso contrário. Considera-se sénior alguém cuja idade é igual ou superior a 65.

3. **Como podemos ler o input?**

   Para ler o inteiro usamos `scanner.nextInt()` e para ler o booleano usamos `scanner.nextBoolean()`.

4. **Como podemos decidir o output?**

   Usamos a instrução *if-else* juntamente com expressões booleanas. Como temos 2 condições temos de juntar ambas.

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("What's your age?");
        int age = scanner.nextInt();

        System.out.println("Do you have a discount coupon?");
        boolean hasCoupon = scanner.nextBoolean();

        if (age >= 65 || hasCoupon) {
            System.out.println("Discount applied");
        } else {
            System.out.println("No discount");
        }
    }
}
```

Consola:

```
What’s your age?
25
Do you have a discount coupon?
true
Discount applied
```

```
What's your age?
67
Do you have a discount coupon?
false
Discount applied
```

```
What's your age?
30
Do you have a discount coupon?
false
No discount
```

Ao contrário do problema anterior, agora temos 2 condições em que basta uma delas ser verdade. No outro problema, ambas as condições tinham de ser verdade. Como tal, neste problema usamos o operador `||`, que significa que uma ou outra expressão é verdadeira.

Verificação de bilhete

Imagina agora, que para pessoas que **não** têm bilhete, estas são redirecionadas para a bilheteira. Queremos escrever um programa que recebe como input se a pessoa tem ou não bilhete (*boolean*). O programa deve imprimir "Please buy a ticket" caso **não** tenha bilhete, ou "You can proceed" caso contrário.

Repartindo o problema em diferentes partes:

1. **Qual é o input?**

   Um booleano que representa se possui um bilhete.

2. **Quais são as condições do output?**

   O output é "Please buy a ticket" caso **não** tenha bilhete, ou "You can proceed" case contrário.

3. **Como podemos ler o input?**

   Para ler o booleano usamos `scanner.nextBoolean()`.

4. **Como podemos decidir o output?**

   Usamos a instrução *if-else* juntamente com a expressão booleana. A expressão deve ser "não tem bilhete".

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Do you have a ticket?");
        boolean hasTicket = scanner.nextBoolean();

        if (!hasTicket) {
            System.out.println("Please buy a ticket");
        } else {
            System.out.println("You can proceed");
        }
    }
}
```

Consola:

```
Do you have a ticket?
false
Please buy a ticket
```

```
Do you have a ticket?
true
You can proceed
```

Para verificar se o valor da variável `hasTicket` é falsa podíamos ter feito `hasTicket == false`, mas uma forma mais simples é usando o **operador de negação** `!`. Este operador devolve o oposto do valor booleano: `true` → `false` e `false` → `true`.

Divisão e resto

Imagina que tens 5 baldes pelos quais queres distribuir igualmente 15 litros de água. Após distribuir a água igualmente, com quantos litros ficará cada balde?

A solução é muito simples, divide-se os 15 litros pelos 5 baldes, $15/5 = 3$. Cada balde ficará com 3 litros.

Agora, e se o número de baldes e a quantidade de litros forem dados como input de um programa que imprime a quantidade de litros com que cada balde fica?

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Read quantity of liters from user
        System.out.println("Enter water quantity in liters");
        int litersOfWater = scanner.nextInt();

        // Read number of buckets from user
        System.out.println("Enter number of buckets");
        int bucketCount = scanner.nextInt();

        int litersPerBucket = litersOfWater / bucketCount;

        System.out.println("Each bucket will have " +
litersPerBucket + " liters of water!");
    }
}
```

Consola:

```
Enter water quantity in liters
15
Enter number of buckets
5
Each bucket will have 3 liters of water!
```

E se tivermos 2 baldes e 5 litros? Seria $5/2 = 2{,}5$ litros.

Consola:

```
Enter water quantity in liters
5
Enter number of buckets
2
Each bucket will have 2 liters of water!
```

Ele imprimiu que cada balde terá 2 litros, em vez de 2,5 litros! Onde foi o restante 1 litro?

Às vezes quando dividimos números inteiros, há um resto. Neste caso sobrou 1 litro. Esse 1 litro é o resto.

Em Java, para obtermos o resto de uma divisão entre inteiros, usamos o operador `%`.

```java
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Read quantity of liters from user
        System.out.println("Enter water quantity in liters");
        int litersOfWater = scanner.nextInt();

        // Read number of buckets from user
        System.out.println("Enter number of buckets");
        int bucketCount = scanner.nextInt();

        int litersPerBucket = litersOfWater / bucketCount;
        int leftoverLiters = litersOfWater % bucketCount;

        System.out.println("Each bucket will have " +
litersPerBucket + " liters of water!");
        System.out.println("There are " + leftoverLiters + "
liters left.");
    }
}
```

Consola:

```
Enter water quantity in liters
5
Enter number of buckets
2
Each bucket will have 2 liters of water!
There are 1 liters left.
```

Podemos ver então, que agora ele diz quantos litros sobraram.

Mas na vida real nós conseguimos dividir litros em frações. No caso, nós esperaríamos que o resultado fosse 2,5 litros e não 2 litros + 1 litro restante.

Repara que a variável que guarda o número de litros que cada balde terá (`litersPerBucket`) é do tipo *integer*. Como vimos anteriormente, o tipo *integer* representa números inteiros, mas 2,5 não é inteiro. Em Java, para representar números não inteiros (número reais) usamos o tipo ***double***. Este tipo permite números inteiros e números não inteiros. Se mudarmos então o tipo da variável para *double*, ficamos com o seguinte:

```java
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Read quantity of liters from user
        System.out.println("Enter water quantity in liters");
        int litersOfWater = scanner.nextInt();

        // Read number of buckets from user
        System.out.println("Enter number of buckets");
        int bucketCount = scanner.nextInt();

        double litersPerBucket = litersOfWater / bucketCount;
        int leftoverLiters = litersOfWater % bucketCount;

        System.out.println("Each bucket will have " +
litersPerBucket + " liters of water!");
        System.out.println("There are " + leftoverLiters + "
liters left.");
    }
}
```

Consola:

```
Enter water quantity in liters
5
Enter number of buckets
2
Each bucket will have 2.0 liters of water!
There are 1.0 liters left.
```

Agora mudou para 2,0 litros. Mostra a casa decimal a 0, mas o resultado da divisão deu um número inteiro. Então o problema tem de estar na divisão.

Em Java, quando divides dois números inteiros, o resultado é sempre um inteiro. O Java não consegue adivinhar frações – ele apenas dá-te a parte inteira da divisão e descarta o resto. Isto é chamado de **divisão inteira**.

Para obter um resultado decimal, pelo menos um dos números tem de ser do tipo *double*. Por exemplo, vamos mudar a quantidade de água em litros para *double*. Não há problema em estarmos a atribuir um valor inteiro a uma variável *double*, pois *double* também pode representar números inteiros.

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Read quantity of liters from user
        System.out.println("Enter water quantity in liters");
        double litersOfWater = scanner.nextInt();

        // Read number of buckets from user
        System.out.println("Enter number of buckets");
        int bucketCount = scanner.nextInt();

        double litersPerBucket = litersOfWater / bucketCount;
        int leftoverLiters = litersOfWater % bucketCount;

        System.out.println("Each bucket will have " +
litersPerBucket + " liters of water!");
        System.out.println("There are " + leftoverLiters + "
liters left.");
    }
}
```

Mas ao tentares compilar este código, vais te deparar do seguinte erro:

```
incompatible types: possible lossy conversion from double to int
```

O erro acontece porque a variável `leftoverLiters` é do tipo *integer* e o resultado da operação de resto que estamos a tentar atribuir (`litersOfWater % bucketCount`), dá origem a um *double*, pois a operação é feita entre um *double* (`litersOfWater`) e um *integer* (`bucketCount`). Operações entre *integers*, dão origem a *integers*. Operações que envolvam pelo menos um *double* dão origem a um *double*.

Trocando então o tipo de `leftoverLiters` para *double*:

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Read quantity of liters from user
        System.out.println("Enter water quantity in liters");
        double litersOfWater = scanner.nextInt();

        // Read number of buckets from user
        System.out.println("Enter number of buckets");
        int bucketCount = scanner.nextInt();

        double litersPerBucket = litersOfWater / bucketCount;
        double leftoverLiters = litersOfWater % bucketCount;

        System.out.println("Each bucket will have " +
litersPerBucket + " liters of water!");
        System.out.println("There are " + leftoverLiters + "
liters left.");
    }
}
```

Consola:

```
Enter water quantity in liters
5
Enter number of buckets
2
Each bucket will have 2.5 liters of water!
There are 1.0 liters left.
```

Agora sim, deu-nos o resultado que queríamos: 2,5 litros!

Repara que, ao contrário de como é em português, o separador decimal é o ponto (.) e não a vírgula (,). Então, em código, sempre que quisermos especificar um número decimal usamos o ponto para separar a parte inteira da parte decimal.

Checkpoint #4

Revê o que aprendeste

- Operadores **lógicos**:
  - `a && b` – a e b
  - `a || b` – a ou b
- Divisão inteira:
  - `14 / 5 = 2`
- Resto de divisão
  - `14 % 5 = 4`

Ciclo while

Pedir um número positivo

Queremos um programa em que é pedido um número inteiro ao utilizador. Se o número for positivo imprime "You entered X" (onde X é o número que o utilizador introduziu) e termina. Porém, se o utilizador não fornecer um número positivo o programa imprime "Please provide a positive number" e pede novamente um número. Ele repete este processo infinitamente até ler um número positivo do utilizador.

Algumas questões que podemos fazer:

Qual é o input?

Um número inteiro.

O que é que o programa deve fazer com o número?

- Se for positivo, o programa imprime "You entered X" (X é o número do utilizador).
- Se não for positivo, o programa imprime "Please provide a positive number" e pede novamente o número. Repete isto até receber um número positivo.

O que é um número positivo?

Um número positivo é um número maior que 0.

O seguinte código faz parcialmente o que queremos.

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Read number from user
        System.out.println("Enter an integer number");
        int number = scanner.nextInt();

        if (number > 0) {
            System.out.println("You entered " + number);
        } else {
            System.out.println("Please provide a positive
number");
        }
    }
}
```

Consola:

```
Enter an integer number
-3
Please provide a positive number
```

O problema é que, depois do utilizador fornecer um número não positivo, o programa para. O objetivo é que volte a perguntar indefinidamente até receber um número positivo.

Uma primeira abordagem, ingénua, seria repetir o código de leitura e verificação do número. Mas isso só não leva a **código repetido** como também não permite que seja infinito. Para este tipo de casos, em Java, podemos usar o **ciclo *while***. O ciclo *while* permite ficar a executar um bloco de código repetidamente enquanto uma dada condição é verdadeira.

Algumas questões que podemos fazer:

O que queremos executar repetidamente?

Ler um número inteiro do utilizador.

Qual é a condição para repetir?

Queremos repetir caso o número não seja positivo.

O que é um número não positivo?

Se um número positivo é maior que 0, um número não positivo é menor ou igual a 0.

```java
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Read number from user
        System.out.println("Enter an integer number");
        int number = scanner.nextInt();

        // Keep asking for number if it's not positive
        while (number <= 0) {
            // Show error message since the number is not positive
            System.out.println("Please provide a positive
number");

            // Read new number from user
            int number = scanner.nextInt();
        }

        /*
            If the program reaches this part of the code, it means
it exited
            the while loop and the provided number is positive.
         */
        System.out.println("You entered " + number);
    }
}
```

Ao tentares rodar o programa vais te deparar do seguinte erro:

```
variable number is already defined in method
main(java.lang.String[])
```

O erro diz que a variável `number` já está definida. E de facto ela já foi definida no início e estamos a defini-la novamente dentro do ciclo *while*. Em vez de definir, o que devemos fazer é mudar o valor da variável.

```java
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Read number from user
        System.out.println("Enter an integer number");
        int number = scanner.nextInt();

        // Keep asking for number if it's not positive
        while (number <= 0) {
            // Show error message since the number is not positive
            System.out.println("Please provide a positive
number");

            // Read new number from user
            number = scanner.nextInt();
        }

        /*
            If the program reaches this part of the code, it means
it exited
            the while loop and the provided number is positive.
         */
        System.out.println("You entered " + number);
    }
}
```

Consola:

```
Enter an integer number
-3
Please provide a positive number
-8
Please provide a positive number
-5
Please provide a positive number
4
You entered 4
```

Como podes ver, se introduzirmos um número não positivo ele pede para fornecer um número positivo indefinidamente até fornecermos um número positivo.

Ciclo *do while*

Se fores analisar o código anterior, a linha que lê o inteiro da consola está repetida, o que não é muito grave, mas dá para facilmente melhorar esse aspeto. O bloco do *while* apenas executa caso e durante a condição seja verdadeira. Neste caso, a condição é o número não ser positivo, mas queremos que execute da primeira vez, quando ainda não tivermos o número. Para isso, podemos usar o ***do while***. O *do while* funciona praticamente da mesma forma que o *while*, com a diferença de que executa a primeira vez e depois repete as execuções enquanto a condição for verdade.

```java
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Enter an integer number");

        int number;

        // Ask for number the first time and keep asking if it's
not positive
        do {
            // Ask for number
            number = scanner.nextInt();

            // Show error message if the number is not positive
            if (number <= 0) {
                System.out.println("Please provide a positive
number");
            }
        } while (number <= 0);

        /*
            If the program reaches this part of the code, it means
it exited
            the while loop and the provided number is positive.
         */
        System.out.println("You entered " + number);
    }
}
```

O resultado é exatamente o mesmo, mas evitamos repetir o código.

Ciclo *for*

Imagina um programa que imprime “Hello, world!” o número de vezes especificadas pelo utilizador. O programa recebe como input um número inteiro. Novamente estamos perante um problema de repetição, neste caso em vez de ser repetir o pedido de input, é repetir o output, que é mais simples. Precisamos então de um ciclo *while*.

Algumas questões a fazer:

O que queremos repetir?

Imprimir “Hello, world!”.

Qual é a condição para continuar a repetir?

Ainda não ter impresso o número de vezes especificadas pelo utilizador. Por exemplo, o utilizador introduz 4, o ciclo deve repetir-se enquanto não tiver impresso 4 vezes.

Então, a condição do ciclo *while* seria algo como “número de vezes repetido inferior a n”, em que *n* é o número dado pelo utilizador. Temos, então, que saber o número de vezes que repetimos o ciclo até agora. Para isso, podemos usar uma variável que conta o número de vezes que repetimos o ciclo.

```java
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Enter an integer number");

        // Count variables
        int total = scanner.nextInt();
        int current = 1;

        // Repeat while current count is less than the desired
repetition count
        while (current <= total) {
            System.out.println("Hello, world!");
            current = current + 1;
        }
    }
}
```

Consola:

```
Enter an integer number
4
Hello, world!
Hello, world!
Hello, world!
Hello, world!
```

Este código está correto e funciona bem, mas, em Java, existe uma forma melhor de fazer um ciclo destes. Neste tipo de casos, em que sabemos exatamente o **início** e o **fim** do ciclo, podemos usar o ciclo *for*. Neste caso ficaria da seguinte forma.

```java
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Enter an integer number");
        int total = scanner.nextInt();

        // Repeat while current count is less than the desired
repetition count
        for (int current = 1; current <= total; current = current
+ 1) {
            System.out.println("Hello, world!");
        }
    }
}
```

O resultado é exatamente o mesmo, mas o código fica mais simples e organizado. As instruções do *for* que estão entre parênteses são:

- `int current = 1` – Definir e inicializar a variável de repetição. É executado assim que o ciclo iniciar.
- `current <= total` – Condição para repetir. É verificado depois de cada iteração. Se a expressão resultar em verdadeiro ele continua para a próxima iteração, caso contrário termina o ciclo.
- `current = current + 1` – Executado após cada iteração para atualizar a variável. Tipicamente a variável é incrementada ou decrementada.

Mas, tradicionalmente, usa-se o nome `i` para a variável do *for*.

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Enter an integer number");
        int total = scanner.nextInt();

        // Repeat while current count is less than the desired
repetition count
        for (int i = 1; i <= total; i = i + 1) {
            System.out.println("Hello, world!");
        }
    }
}
```

Outra coisa que podemos simplificar é a incrementação da variável. Em Java, podemos abreviar uma expressão `x = x + y` para `x += y`. Neste caso, ficaria `i += 1`. Mas, neste caso específico em que estamos a somar 1, podemos simplificar ainda mais com o operador de incrementação `++`, e então ficaria `i++`. O operador `+=` também tem equivalentes para subtração, multiplicação e divisão. O operador `++` tem equivalente para subtração.

Sintaxe	Equivalente a
`x += y`	`x = x + y`
`x -= y`	`x = x - y`
`x *= y`	`x = x * y`
`x /= y`	`x = x / y`
`x++`	`x = x + 1`
`x--`	`x = x - 1`

```java
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Enter an integer number");
        int total = scanner.nextInt();

        // Repeat while current count is less than the desired
repetition count
        for (int i = 1; i <= total; i++) {
            System.out.println("Hello, world!");
        }
    }
}
```

Instrução *continue*

Imagina agora um programa que aceita como input um número inteiro *n*. O programa fará a soma de todos os números pares até *n*, imprimindo cada número que foi somado, e no fim imprime o resultado. Por exemplo, o utilizador introduz o número 7, o programa deve imprimir o seguinte:

```
7
2
4
6
Sum: 12
```

Questões que podemos fazer:

O que o programa recebe como input?

Um inteiro. Devemos usar então `scanner.nextInt()`.

O programa irá somar vários números até um dado ponto, qual é a condição para continuar a somar?

Queremos que o programa some os números até ao número dado pelo utilizador. Portanto, a condição é que o número atual tem de ser menor ou igual ao número dado pelo utilizador.

Como verificamos se um dado número é par?

Um número par é qualquer número que seja divisível por 2. Ser divisível por 2 significa que o resto da divisão por 2 é 0. Vimos, anteriormente, que para obter o resto da divisão usamos o operador `%`. Então, para um número ser par, `x % 2` tem de ser igual a 0.

Dada esta informação, chegamos a algo semelhante a isto:

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Enter an integer number");
        int max = scanner.nextInt();
        int sum = 0;

        for (int i = 1; i <= max; i++) {
            if (i % 2 == 0) {
                System.out.println(i);
                sum += i;
            }
        }

        System.out.println("Sum: " + sum);
    }
}
```

Consola:

```
7
2
4
6
Sum: 12
```

Se fores analisar o código, consegues ver que temos um for e dentro dele temos um *if*, e isto pode ser simplificado para leitura mais fácil. Quando estamos num ciclo (*for*, *while* ou *do while*) podemos pular a iteração atual usando a instrução ***continue***. Neste caso, se o número for ímpar, queremos saltar para o próximo número. Se for par continuamos a nossa lógica de somar e imprimir. Se um número par tem resto **igual 0**, um número ímpar tem resto **diferente de zero**. Além disto, podemos também omitir as chavetas ({ }) do *if* e escrever tudo na mesma linha. Quando temos apenas 1 instrução/linha, podemos escrever o *if* sem chavetas, e se for uma instrução curta podemos escrever tudo na mesma linha.

```java
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Enter an integer number");
        int max = scanner.nextInt();
        int sum = 0;

        for (int i = 1; i <= max; i++) {
            if (i % 2 != 0) continue;

            System.out.println(i);
            sum += i;
        }

        System.out.println("Sum: " + sum);
    }
}
```

O resultado é exatamente o mesmo, mas o código fica mais limpo.

Instrução *break*

Tendo o programa anterior como base, vamos acrescentar-lhe um novo comportamento. O utilizador, agora, além do número máximo, também fornece o número mínimo.

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Enter the minimum");
        int min = scanner.nextInt();
        System.out.println("Enter the maximum");
        int max = scanner.nextInt();

        int sum = 0;

        for (int i = min; i <= max; i++) {
            if (i % 2 != 0) continue;

            System.out.println(i);
            sum += i;
        }

        System.out.println("Sum: " + sum);
    }
}
```

Consola:

```
Enter the minimum
4
Enter the maximum
9
4
6
8
Sum: 18
```

Agora, queremos que o programa pare de somar se encontrar um número que seja múltiplo de 10. Atualmente, estamos a usar o *continue* para saltar a iteração atual, mas é possível saltar para fora do ciclo por completo, usando a instrução ***break***.

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Enter the minimum");
        int min = scanner.nextInt();
        System.out.println("Enter the maximum");
        int max = scanner.nextInt();

        int sum = 0;

        for (int i = min; i <= max; i++) {
            // Skip if multiple of 2
            if (i % 2 != 0) continue;

            // Exit loop if multiple of 10
            if (i % 10 != 0) break;

            System.out.println(i);
            sum += i;
        }

        System.out.println("Sum: " + sum);
    }
}
```

Consola:

```
Enter the minimum
4
Enter the maximum
15
4
6
8
Sum: 18
```

Checkpoint #5

Revê o que aprendeste

- Ciclo ***while***:

```
int count = 0;
while (count < 5) {
    System.out.prinln(count);
    count++;
}
```

- Ciclo ***do while***:

```
int count = 0;
do {
    System.out.prinln(count);
    count++;
} while (count < 5);
```

- Ciclo ***for***:

```
for (int i = 0; i < 5; i++) {
    System.out.prinln(i);
}
```

- Instruções ***break*** e ***continue***

Introdução à Programação Orientada a objetos

Classes

Já vimos que podemos guardar dados em variáveis, como nome, idade, etc. Imagina agora que queremos guardar o nome e idade de 3 diferentes alunos. Depois queremos imprimir uma mensagem do aluno a apresentar-se dizendo o nome e idade.

```
public class Main {
    public static void main(String[] args) {
        String name1 = "John";
        int age1 = 20;
        System.out.println("Hi, my name is " + name1 + " and I'm "
+ age1 + " years old.");


        String name2 = "Alice";
        int age2 = 24;
        System.out.println("Hi, my name is " + name2 + " and I'm "
+ age2 + " years old.");

        String name3 = "Bob";
        int age3 = 19;
        System.out.println("Hi, my name is " + name3 + " and I'm "
+ age3 + " years old.");
    }
}
```

Podemos ver algum código repetido e que é inconveniente estarmos a declarar as variáveis para o nome e a idade repetidamente, para cada aluno.

É aqui que programação orientada a objetos brilha! Como? Com o uso de classes!

De modo geral, uma classe é um conjunto de coisas que têm propriedades em comum. Em programação, no fundo, classes partem do mesmo conceito.

Vamos fazer uma analogia com o mundo real. Nós usamos o conceito de classes no mundo real o tempo todo sem nos apercebermos. Por exemplo, observa a seguinte imagem:

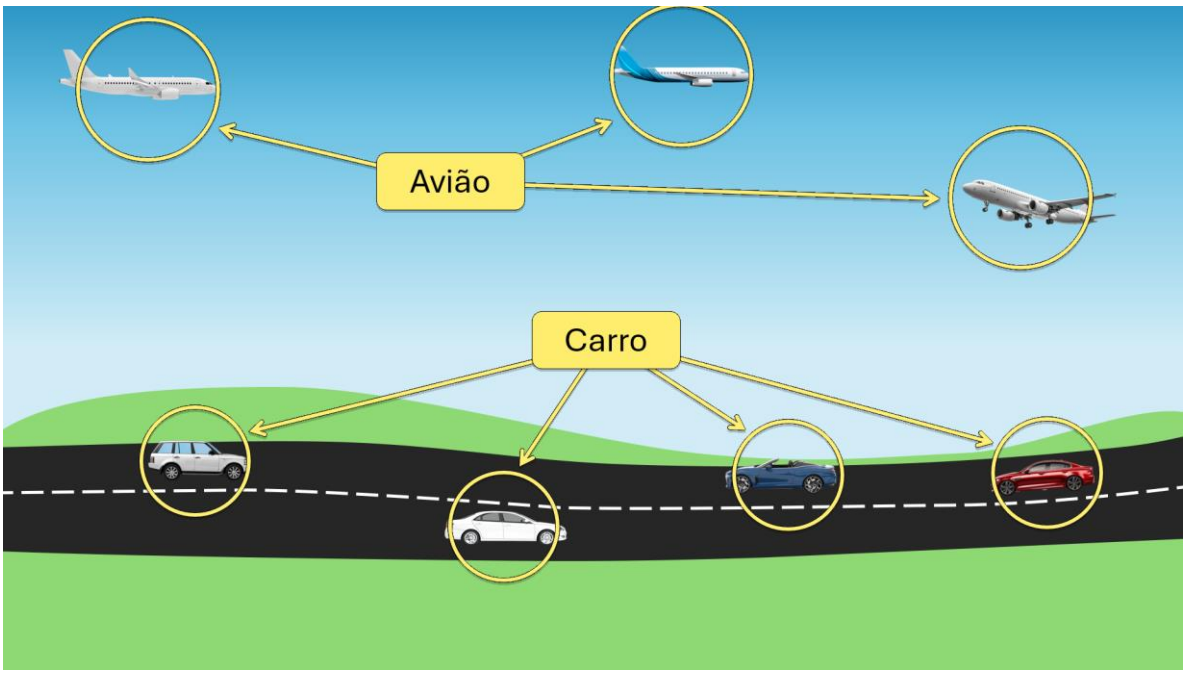


Figura 9 - Analogia a classes

A imagem tem vários objetos, que podemos agrupar em carros e aviões. Este exercício de identificar os objetos, apesar de trivial, está a pôr em prática o conceito de classes. Identificámos as propriedades dos objetos e agrupámos os que partilham propriedades. Mas existem outras propriedades como cor e tamanho que variam.

A **classe carro** tem rodas, janelas e anda no chão. A **classe avião** tem asas, motores e voa no céu. Depois cada carro e avião individual têm características diferentes, como a cor e o tamanho. Cada carro individual é o que chamamos de uma **instância da classe carro**, e vice-versa para a classe avião. As características que diferem entre cada carro (ex.: marca, tamanho e cor) são o que chamamos de **atributos**.

Para definirmos uma classe em Java, criamos um ficheiro novo com o nome da classe.

```
public class Car {
    String brand;
    String color;
    int maxSpeed;
}
```

Esta classe tem 3 atributos: `brand` (marca), `color` (cor) e `maxSpeed` (velocidade máxima).

Voltando ao nosso problema anterior dos alunos. Cada aluno tem um nome (*string*) e idade (*integer*).

```
public class Student {
    String name;
    int age;
}
```

Repara nos atributos `name` e `age`.

Para criarmos uma instância nova de um aluno, usamos o operador ***new***.

```
new Student();
```

Depois, provavelmente vamos querer guardar esta instância numa variável.

```
Student student = new Student();
```

Repara que o tipo da variável é `Student`.

Aplicando, então, aos alunos todos...

```
public class Main {
    public static void main(String[] args) {
        Student student1 = new Student();
        student1.name = "John";
        student1.age = 20;
        System.out.println("Hi, my name is " + student1.name + "
and I'm " + student1.age + " years old.");

        Student student2 = new Student();
        student2.name = "Alice";
        student2.age = 24;
        System.out.println("Hi, my name is " + student2.name + "
and I'm " + student2.age + " years old.");

        Student student3 = new Student();
        student3.name = "Bob";
        student3.age = 19;
        System.out.println("Hi, my name is " + student3.name + "
and I'm " + student3.age + " years old.");
    }
}
```

Métodos

Repara que no exemplo anterior dos alunos, estamos a repetir a parte de imprimir na consola. Quando temos ações que variam pouco, podemos usar **métodos**. Um método é um pedaço de código que podemos executar, chamando-o com o nome dele.

```
public class Student {
    String name;
    int age;

    void introduce() {
        System.out.println("Hi, my name is " + this.name + " and
I'm " + this.age + " years old.");
    }
}
```

O código acima define uma classe `Student` com um método `introduce`. Este método usa os atributos `name` e `age` para um imprimir a mensagem de introdução do aluno. Repara no uso da palavra-chave ***this***, que permite aceder à **instância**.

Depois podemos chamar o método da seguinte forma.

```
public class Main {
    public static void main(String[] args) {
        Student student1 = new Student();
        student1.name = "John";
        student1.age = 20;
        student1.introduce();

        Student student2 = new Student();
        student2.name = "Alice";
        student2.age = 24;
        student2.introduce();

        Student student3 = new Student();
        student3.name = "Bob";
        student3.age = 19;
        student3.introduce();
    }
}
```

O código faz o mesmo, mas ficou mais simples e mais legível!

Parâmetros de métodos

Já vimos como definir um método. Mas, assim como programas, métodos também podem receber dados. Estes dados são recebidos através de **parâmetros**, que são como variáveis.

Imagina que queremos modificar o método que tínhamos anteriormente (para o aluno se apresentar), mas que agora queremos que o aluno se apresente a outro aluno. O nome desse outro aluno é dado por um parâmetro `otherName`.

```
public class Student {
    String name;
    int age;

    void introduceTo(String otherName) {
        System.out.println("Hi" + otherName + ", my name is " +
this.name + " and I'm " + this.age + " years old.");
    }
}
```

Depois, quando formos chamar o método, passamos um **argumento**, neste caso uma string.

```
public class Main {
    public static void main(String[] args) {
        Student student = new Student();
        student.name = "John";
        student.age = 20;
        student.introduceTo("Alice");
    }
}
```

Consola:

```
Hi Alice, my name is John and I’m 25 years old.
```

Tipo de retorno de métodos

Vimos que métodos podem receber dados, mas eles podem também enviar de volta algum dado. Para isso, temos de especificar o **tipo de retorno**. Até ao momento usámos ***void***, que simplesmente indica que o método não devolve nada.

Neste caso, em vez de imprimirmos a mensagem, queremos devolvê-la.

```
public class Student {
    String name;
    int age;

    String getIntroductionTo(String otherName) {
        return "Hi" + otherName + ", my name is " + this.name + "
and I'm " + this.age + " years old.";
    }
}
```

Repara no uso da instrução ***return***, que devolve o valor que queremos.

Depois, podemos guardar numa variável e imprimir.

```
public class Main {
    public static void main(String[] args) {
        Student student = new Student();
        student.name = "John";
        student.age = 20;
        String introduction = student.getIntroductionTo("Alice");
        System.out.println(introduction);
    }
}
```

Consola:

```
Hi Alice, my name is John and I’m 25 years old.
```

O resultado é o mesmo, mas já não é a classe `Student` que imprime.

Construtores

Analisando o código de antes, podemos notar que estamos a atribuir valores aos atributos da seguinte forma:

```
public class Main {
    public static void main(String[] args) {
        Student student = new Student();
        student.name = "John";
        student.age = 20;
        String introduction = student.getIntroductionTo("Alice");
        System.out.println(introduction);
    }
}
```

Porém, podemos simplificar isto. Quando queremos inicializar uma instância com valores iniciais, definimos um **construtor**. Um construtor é uma função de uma classe que a inicializa.

```
public class Student {
    String name;
    int age;

    Student(String name, int age) {
        this.name = name;
        this.age = age;
    }
}
```

Como podes observar, nosso caso o construtor recebe o nome e idade e atribui-os aos atributos.

Para inicializar fazemos da seguinte forma:

```
public class Main {
    public static void main(String[] args) {
        Student student = new Student("John", 25);
        String introduction = student.getIntroductionTo("Alice");
        System.out.println(introduction);
    }
}
```

O código ficou mais simples!

Objetos vs primitivos

Instâncias de classe chamamos de **objetos**, que em contraste com tipos **primitivos** (*integer*, *double*...), não são valores simples como números. São estruturas que podem ser complexas e conter vários atributos.

Exemplos de variáveis primitivas

```
int x = 5;
double y = 3.14;
```

Stack vs heap

Por objetos serem muito mais complexos, eles não são guardados no mesmo sítio que primitivos. Primitivos são guardados num lugar da memória chamado de ***stack***, que consiste em memória contínua. Os objetos são guardados no ***heap***. O heap não é contínuo. As variáveis no *stack* apontam para objetos no *heap*.

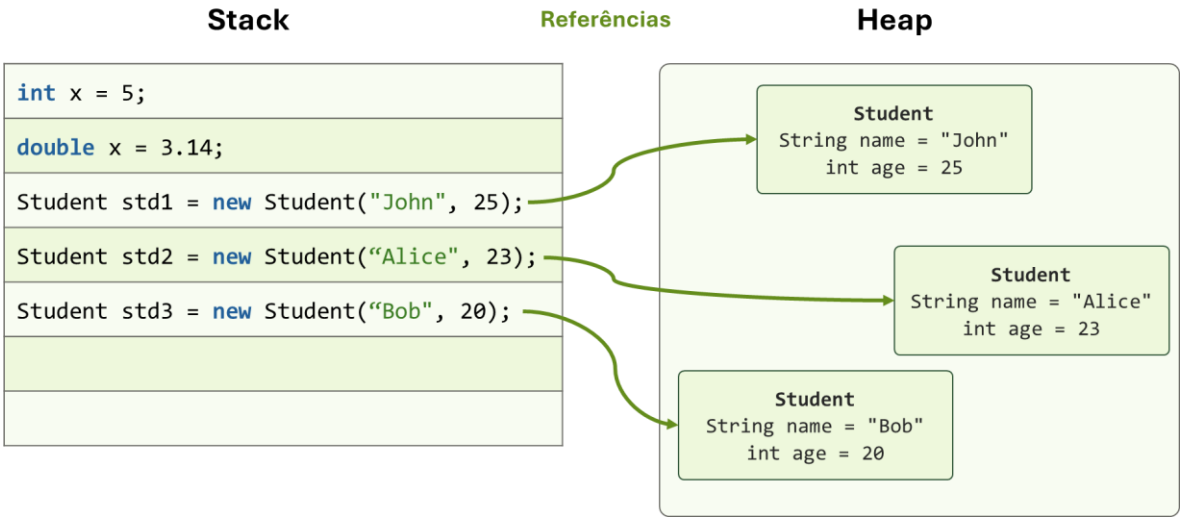


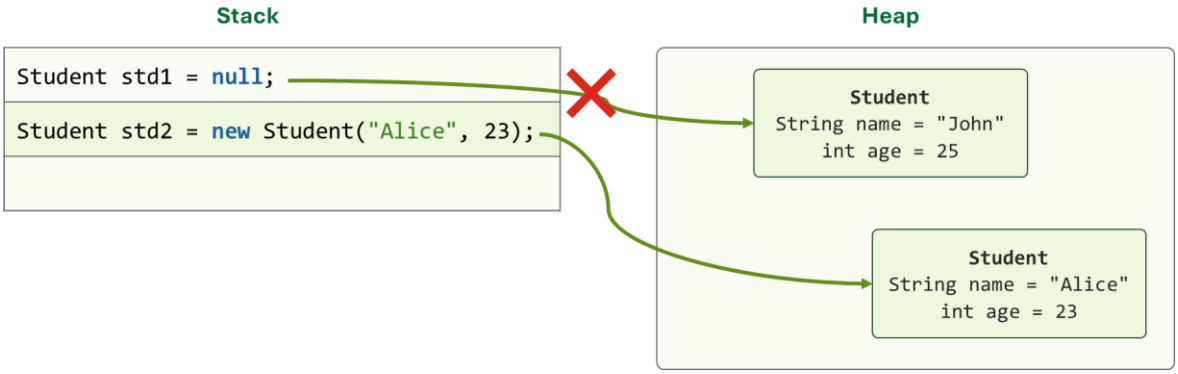
Figura 10 - Stack e heap

Garbage Collection

Sempre que instanciamos um objeto novo e atribuímos a uma variável, ele é alocado no *heap*. Se atribuirmos o valor `null` à variável, a variável deixa de apontar para qualquer objeto. Mas, ao fazermos isto, o objeto no heap tem de ser destruído, caso contrário iríamos encher o *heap* indefinidamente e encher a memória do computador. Em algumas linguagens temos de manualmente remover objetos, mas no caso do Java é feito automaticamente pelo ***Garbage Collector***!

Na imagem abaixo, podemos ver 2 variáveis (`std1` e `std2`) que inicialmente apontam para objetos do tipo `Student`. Depois, `std1` é atribuído o valor `null` e deixa de apontar para o objeto `Student`. Então, depois, o *Garbage Collector* entrará em ação e destruirá o objeto. Este processo chama-se ***Garbage collection***.

```
Student std1 = new Student("John", 25);
Student std2 = new Student("Alice", 23);
std1 = null;
```



Mais está por vir!

Referência

Tipos de dados

Primitivos

Tipo	Descrição
`byte`	Números inteiros de 8 bits (1 byte) (-128 a 127)
`short`	Números inteiros de 16 bits (-32 768 a 32 767)
`int`	Números inteiros de 32 bits (-2 147 483 648 a 2 147 483 647)
`long`	Números inteiros de 64 bits (-9 223 372 036 854 775 808 a 9 223 372 036 854 775 807)
`float`	Números fracionários de 32 bits
`double`	Números fracionários de 64 bits
`boolean`	Valor booleano (`true` ou `false`)
`char`	Um único carácter ou valor ASCII

Não-primitivos

Tipo	Descrição
`String`	Sequência de caracteres (texto).

Operadores

Aritmética

Operador	Sintaxe	Exemplo	
Adição	`x + y`	`5 + 2` → `7`	
Subtração	`x - y`	`5 - 2` → `3`	
Multiplicação	`x * y`	`5 * 2` → `10`	
Divisão	`x / y`	`5 / 2` → `2`	`5 / 2.0` → `2.5`
Resto da divisão	`x % y`	`5 % 2` → `1`	

Comparação

Operador	Sintaxe	Significado
Igual	`x == y`	x igual a y
Diferente	`x != y`	x diferente de y
Maior que	`x > y`	x maior que y
Maior ou igual	`x >= y`	x maior ou igual a y
Menor que	`x < y`	x menor que y
Menor ou igual	`x <= y`	x menor ou igual a y

Lógica

Operador	Sintaxe	Significado
“E” lógico	`a && b`	a e b
“Ou” lógico	`a || b`	a ou b

Outros

Sintaxe	Equivalente a
`x += y`	`x = x + y`
`x -= y`	`x = x - y`
`x *= y`	`x = x * y`
`x /= y`	`x = x / y`
`x++`	`x = x + 1`
`++x`	`x = x + 1`
`x--`	`x = x - 1`
`--x`	`x = x - 1`

Controlo

Instrução *if*

```
if (<condição>) {
    <se verdadeiro>
} else if (<condição>) {
    <se verdadeiro>
} else {
    <se falso>
}
```

Ciclo *while*

```
while (<condição>) {
    <enquanto verdadeiro>
}
```

Ciclo *do while*

```
do {
    <enquanto verdadeiro>
} while (<condição>);
```

Instruções break e continue

- `continue` – saltar a iteração atual.
- `break` – sair fora do ciclo.

Input/Output

Ler input

```
Scanner scanner = new Scanner(System.in);
scanner.nextInt(); // Ler um integer
scanner.nextDouble(); // Ler um double
scanner.nextBoolean(); // Ler um boolean
scanner.nextLine(); // Ler uma linha (string)
```

Imprimir output

```
System.out.prinln("Hello, world!");
```

Métodos

Modificadores

- *default*
- *private*
- *protected*
- *public*

Sem retorno

```
// Define the method
<modificador> [static] void <nome>(<parametros>) {
    <corpo>
}

// Call the method
<nome>(<parametros>);
```

Exemplo:

```
void greet(String name, int age) {
    System.out.println("Hello, my name is " + name + " and I'm " +
age + " years old.");
}
```

Com retorno

```
// Define the method
<modificador> [static] <tipo> <nome>(<parametros>) {
    <corpo>
}

// Call the method
<nome>(<parametros>);
```

Exemplo:

```
int triple(int x) {
    return x * 3;
}

int x = triple(4);
System.out.println(x); // Prints 12 (4 * 3)
```